



POLITECNICO
MILANO 1863



ITECNICO
LANO 1863

architettura dei calcolatori e sistemi operativi

assembler

da C ad assembler

RISC V - capitolo 2 P&H

compilazione

come si passa da un linguaggio di alto livello a uno di basso livello?

- ovvero, come funziona un compilatore, che effettivamente traduce un linguaggio a più alto livello in uno a più basso livello?
- come esempio di passaggio da alto a basso livello
 - linguaggio C come alto livello
 - linguaggio assembler RISC V come basso livello
- discuteremo una possibile traduzione / compilazione dei vari costrutti del linguaggio C
 - utile concettualmente
 - mette progressivamente in evidenza le differenze tra alto e basso livello, che cosa avviene veramente nella macchina



struttura generale

il processo di traduzione da linguaggio sorgente di alto livello a linguaggio macchina è diviso in quattro fasi

- **analisi sintattica** e **trattamento errori** - ling. sorgente **alto livello**
- **generazione** del codice assembly - ling. macchina **simbolico (assembler)**
- **ottimizzazione** del codice assembly - facoltativa
- **assemblaggio** e **collegamento** - ling. macchina **numerico**

di solito si vede il risultato: codice macchina eseguibile
volendo si possono avere le rappresentazioni intermedie

- albero sintattico del programma
- codice macchina simbolico, non ottimizzato
- eventuali altre forme intermedie ...

spesso si può scegliere il modello di processore per cui tradurre
e così ottenere codice macchina numerico *ad hoc*



analisi sintattica e generazione di codice

analisi sintattica

- verifica se il programma è sintatticamente corretto
- segnala e corregge errori

generazione di codice

- il programma viene tradotto in linguaggio macchina
- il codice macchina prodotto è in forma simbolica
- non numerica ossia non eseguibile direttamente

linguaggio sorgente di alto livello (C ANSI)

```
typedef long long int LONG
LONG a          // intero a 64 bit
LONG b = 3      // idem, val ini 3
a = b + 1
```

```
A: .zero 8 // LONG a (64 bit)
B: .dword 3 // LONG b = 3 (64 bit)
la t0, B // load address
ld t1, (t0) // load dword
addi t1, t1, 1 // add immediate
la t0, A // load address
sd t1, (t0) // store dword
```

ling. macchina simbolico (RISC-V RV64i)
il tipo intero lungo ingombra 8 byte



assemblaggio e collegamento

assemblaggio

- risolve simboli
assemblatore
 - indirizzo di memoria
 - etichetta d'istruzione
 - spiazzamento
- l'istruzione simbolica viene tradotta in codifica numerica

collegamento (*linking*)

- unisce più moduli eseguibili
- per esempio programma e librerie standard (IO ecc)

```
A: .zero 8
B: .dword 3
la t0, B
ld t1, (t0)
addi t1, t1, 1
la t0, A
sd t1, (t0)
```

*ling. macchina
simbolico*

*tipo intero lungo
ingombra 8 byte*

codici operativi

ld	1010...
addi	1100...
sd	1010...

tab. dei simboli

A	1010...
B	1101...

```
1010 .....
1100 .....
1010 .....
.....
```

ling. macchina numerico



istruzioni vs pseudo istruzioni

sono istruzioni del linguaggio
assemblatore che hanno una
corrispondenza uno-a-uno
con le istruzioni della ISA

`la t0, A`

pseudoistruzione

alzano il livello di astrazione
per l'utente e sono più
intuitive e generalizzabili,
ma devono essere **tradotte**
in modi differenti secondo
l'architettura di processore
usata (componendo più
istruzioni della ISA)

**vedi APPROFONDIMENTO
in coda alle slide**

`auipc t0, %pcrel_hi(A)`
`addi t0, t0, %pcrel_lo(A)`

istruzioni



istruzione *ecall* – invocazione del SO

- istruzione di chiamata (invocazione) di sistema operativo (SO), che fornisce un insieme di funzioni di servizio
- per richiedere una funzione di servizio (simulatore RARS):
 - codice di chiamata (numero servizio) nel registro *a7*
 - argomenti nei registri *a0*, *a1*, ..., *fa0*, *fa1*, ... (secondo il servizio)
 - istruzione macchina ***ecall*** (senza argomenti)
- il servizio inserisce l'eventuale risultato nel registro *a0*

Table of Available Services

Name	Number	Description	Inputs	Ouputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintFloat	2	Prints a floating point number	fa0 = float to print	N/A
PrintDouble	3	Prints a double precision floating point number	fa0 = double to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Reads an int from input console	N/A	a0 = the int
Exit2	93	Exits the program with a code	a0 = the number to exit with	N/A



Hello, World!

```
#include <stdio.h>
```

```
void main() {  
    printf("Hello, World!");  
    return;  
}
```

```
Hello, World!
```

```
-- program is finished running (-1) --
```

The screenshot displays a debugger interface with three main panels: Text Segment, Labels, and Data Segment.

Text Segment: A table showing assembly instructions with columns for Bkpt, Address, Code, Basic, and Source.

Bkpt	Address	Code	Basic	Source
	0x00400000	0x00400893	addi x17,x0,4	5: MAIN: li a7, 4 # se...
	0x00400004	0x0fc10517	auipc x10,0x0000fc10	6: la a0, OUT_STRING
	0x00400008	0xffc50513	addi x10,x10,0xffff...	
	0x0040000c	0x00000073	ecall	7: ecall # in...
	0x00400010	0x05d00893	addi x17,x0,0x0000005d	8: EXIT: li a7, 93 # se...
	0x00400014	0xffff0013	addi x10,x0,0xffffffff	9: li a0, -1 # co...
	0x00400018	0x00000073	ecall	10: ecall # in...

Labels: A table showing labels and their addresses.

Label	Address
helloworld.asm	
MAIN	0x00400000
EXIT	0x00400010
OUT_STRING	0x10010000

Data Segment: A table showing memory values at different addresses.

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6c65480a	0x202c6f6c	0x6c72657	0x000a2164	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

At the bottom, there are navigation buttons and checkboxes for Hexadecimal Addresses, Hexadecimal Values, and ASCII.

```
1  # traduzione in assembler RISC V del programma Hello World
2
3  OUT_STRING:      .data          # sezione di dichiarazione dati
4                  .asciz          "\nHello, World!\n"
5  MAIN:           .text          # sezione di dichiarazione codice
6                  li    a7, 4     # servizio di stampa stringa
7                  la    a0, OUT_STRING
8                  ecall          # invocazione del servizio di S0
9  EXIT:           li    a7, 93    # servizio di terminazione
10                  li    a0, -1   # codice di terminazione
11                  ecall          # invocazione del servizio di S0
```

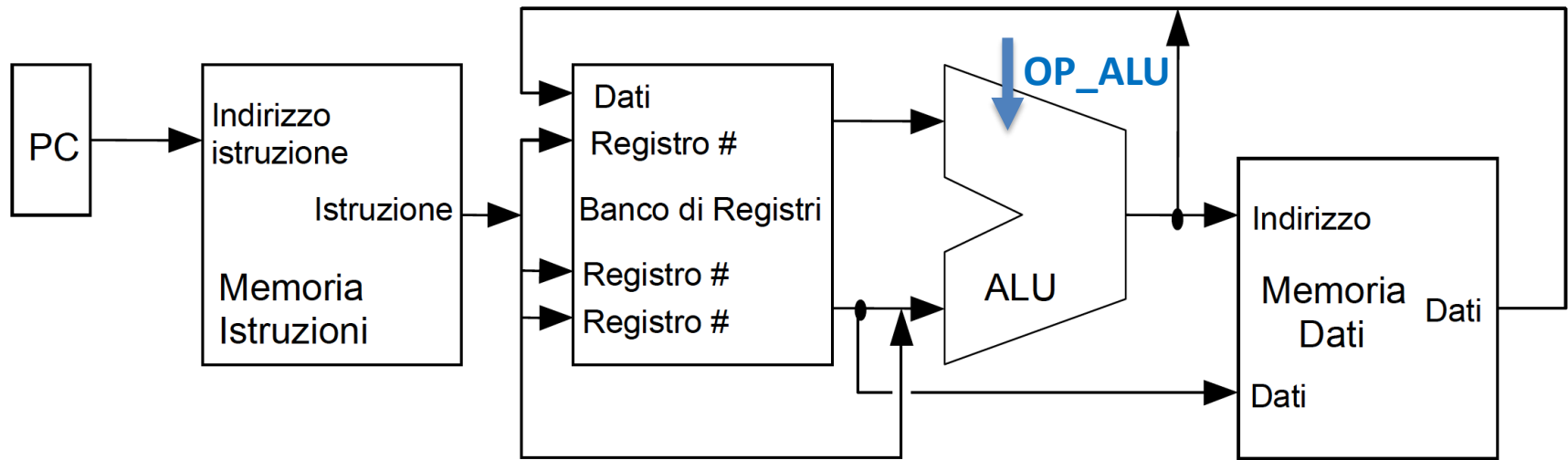


considerazioni generali

- qui si illustra un modello di compilazione per **tradurre da linguaggio C a linguaggio macchina RISC-V**, ispirato a
 - *Instruction Set Architecture* (ISA) **RV64i** (aritmetica intera a 64 bit)
 - *Application Binary Interface* (ABI) di **GCC** (compilatore C di Linux)
- **presuppone di conoscere RISC-V: banco di registri, classi di istruzioni, modi di indirizzamento e organizzazione del sottoprogramma (chiamata, rientro e passaggio parametri)**
- consiste in vari insiemi di convenzioni e regole per
 - segmentare il programma
 - dichiarare le variabili
 - usare (leggere e scrivere) le variabili
 - rendere le strutture di controllo
- Quello che faremo non attuerà necessariamente la traduzione più efficiente (.... non è un corso di compilatori)
- sono possibili varie ottimizzazioni *ad hoc* del codice



struttura base del processore RISC-V



```
add      s0, s1, s2
```

```
# s0 = s1 + s2
```

```
add      s0, s0, s0
```

```
# s0 = s0 + s0
```

```
and      s1, s1, s0
```

```
# s1 = s1 & s0
```



SO e modello di memoria del processo

per il SO il *processo* (cioè la copia in memoria del codice binario del nostro programma scritto in linguaggio macchina) ha, tipicamente, tre segmenti essenziali

- **codice** (o **testo**) istruzioni della funzione *main* e delle funzioni *utente*
- **dati** valori di variabili *globali e statiche* (cioè valori di variabili il cui tempo di vita coincide essenzialmente con il “tempo di vita” dell’intero programma, seppur con regole di visibilità e momenti di “nascita” differenti) e *dinamiche* (create via *malloc*, quindi con “tempo di vita” deciso dal programmatore)
- **pila** aree(record) di attivazione di funzioni, contenenti *indirizzi*, *parametri*, eventuali *registri salvati* e variabili *locali*

Codice e **dati** sono **segmenti** dichiarati nel programma.

Il segmento di **pila** viene creato al lancio del processo.

processi grandi o sofisticati hanno facoltativamente

- due o più segmenti codice o dati
- segmenti di dati condivisi
- segmenti di libreria dinamica (per esempio *libstdc++*, *libc* e altre)
- e altre peculiarità ... (per esempio segmenti di *pila di thread*)

questo modello di memoria è valido in generale



modello di memoria dei programmi RISCV64i – testo P&H

max 64 bit di indirizzo – 38 bit

modello di memoria da 38 bit di
indirizzo: 256 GiB

la parte superiore dello spazio virtuale
di memoria resta
inutilizzata dal processo

Agli indirizzi virtuali maggiori c'è il SO

*In generale, gli indirizzi di inizio (di impianto dei segmenti) dipendono dall'ABI
Anche i 4MiB riservati possono essere locati, a seconda dell'ABI, sotto il seg. di testo o sopra il seg. di stack*

$(2^{28} - 4 \cdot 2^{20})$ B: (256 MiB - 4 MiB)

$4 \cdot 2^{20}$ B - 0: 4 MiB riservato,
conterranno la lista delle variabili d'ambiente +
argv[...] del main

Sullo stack: indirizzi di dword

SP → 0000 003f ffff ffff_{esa}



Nel seg. dati: indirizzi di byte

0000 0000 1000 0000_{esa}

Nel seg. di testo: indirizzi di word

PC → 0000 0000 0040 0000_{esa}

0

RISC-V (con ISA RV64i) fa uso di indirizzi da 64 bit

indirizzi rappresentabili con 16 cifre esadecimali (hex)
(per comodità divisi in 4 gruppi da 4 cifre o in 2 gruppi da 8 cifre)

max spazio mem: 2^{64} byte = 4 Gi Gi byte ($1 \text{ Gi} = 2^{30} \approx 10^9 = 1 \text{ G}$)



dichiarare i segmenti tipici

gli indirizzi di impianto dei segmenti sono virtuali (non fisici)

<pre>// var. glob. ... // parti exec main (...) { // corpo ... }</pre>	<pre>// segmento dati // ind. iniz. 0x 0000 0000 1000 0000 .data ... // dichiarazioni var. globali ... // segmento codice (testo) // ind. iniz. 0x 0000 0000 0040 0000 .text MAIN: ... // istruzioni macchina ...</pre>
--	--

non occorre dichiarare esplicitamente il segmento di pila (*stack*), poiché esso inizia implicitamente a un indirizzo dato (p. es. 0000 003f ffff fff0) e viene esteso verso gli indirizzi minori (ossia verso l'indirizzo 0); similmente le variabili dinamiche non sono dichiarate esplicitamente, ma vengono aggiunte (a indirizzi crescenti) al segmento dati durante l'esecuzione del programma, ogniqualvolta si usa la funzione *malloc* del C





POLITECNICO
MILANO 1863

variabili

considerazioni generali

- per tradurre da linguaggio sorgente, p. es. C, a linguaggio macchina, p. es. RISC-V, bisogna definire un modello di architettura *run-time* per memoria e processore
- le convenzioni del modello run-time comprendono
 - collocazione e ingombro delle diverse classi di variabile
 - nomenclatura e destinazione di uso dei registri del processore
- il modello di architettura run-time consente interoperabilità tra porzioni di codice di provenienza differente, come per esempio codice utente e librerie standard precompilate
- esempio tipico in linguaggio C è la libreria standard di I/O



variabili e memoria

mentre il C ha costrutti di programmazione e variabili con tipo, il linguaggio *assembler* ha istruzioni, memoria interna e memoria esterna (indirizzabile al byte)

- **interna** alla CPU: registri

- **esterna** alla CPU: RAM

⇒ dovremo trovare una rappresentazione adeguata delle variabili in memoria

- ciascuna variabile corrisponde a un indirizzo di memoria che contiene il valore della variabile
- esempio: la variabile **a** corrisponde all'indirizzo 0x 100; se la variabile è inizializzata a 1 (in base dieci) la cella di indirizzo 0x 100 contiene il valore 1



dimensione delle variabili

tipi di dato

i linguaggi ad alto livello come il C permettono di definire variabili con associato un tipo:

- caratteri, interi (corti, normali e lunghi), stringhe, reali (virgola mobile – *floating point*) a singola o doppia precisione, ecc
- ciascuno di questi tipi ha una certa taglia: **sizeof** (`tipo`)
 - carattere: 8 bit; intero corto: 16 bit; normale: 32 bit; lungo: 64 bit
 - reale: 32 o 64 bit; stringa: $8 \times (\text{lunghezza della stringa} + 1)$ bit

mentre la “taglia” di un dato è abbastanza trasparente all’utente in linguaggi di alto livello ... in linguaggio macchina ogni cosa va esplicitata

- bisogna conoscere la taglia di ciascun tipo di dato
- e avere abbastanza celle di memoria per ogni variabile di quel tipo



ingombro variabili

- l'unità di memoria minima indirizzabile è il singolo byte
- in C sotto Linux i diversi tipi di variabile ingombrano così

```
sizeof (char)           = 1 byte (b)
sizeof (short int)      = 2 byte o mezza parola (h - half)
sizeof (int)             = 4 byte o parola (w - word)
sizeof (long int)       = 8 byte o parola doppia (d - dword)
sizeof (long long int)  = come long int (d - dword)
sizeof (array)          = somma degli ingombri degli elementi
sizeof (struct)         = somma degli ingombri dei campi
```

- in C sotto Windows *long int* ingombra 4 byte (come *int*)
e *long long int* ingombra 8 byte
- *long long int* ingombra sempre 8 byte (Linux e Windows) !
- per il tipo reale (*float* e *double*) vedi lo standard IEEE 754
(qui non si considera il tipo reale)
- chiameremo sempre *LONG* il tipo intero a 64 bit !



operare sulle variabili

per operare su una variabile (tipicamente globale) in assembler:

1. caricare la variabile in un registro (*ld* e sue varianti *w*, *h*, *b*)
2. eseguire le operazioni prendendo il dato dal registro
3. riscrivere la variabile in memoria (*sd*, e sue varianti *w*, *h*, *b*)

Nota Bene (si spiega più estesamente in seguito): solitamente la variabile globale è allocata in memoria, dunque i passi 1, 2, 3 sono necessari ogniqualevolta si opera sulla variabile; invece, solitamente la variabile locale è allocata in un registro, dunque i passi 1 e 3 sono evitabili (ma non sempre – dettagli in seguito).



esempio

programma C

```
typedef long long int LONG
```

```
LONG a, b, c // interi a 64 bit
```

```
a = b + c
```

mappatura delle variabili intere (8 byte) in memoria

```
A: .dword           # a: indirizzi 100dec...107dec
```

```
B: .dword           # b: indirizzi 108dec...115dec
```

```
C: .dword           # c: indirizzi 116dec...123dec
```

traduzione in assembler RISC V

```
ld   t0, 108(zero)    # carica var. b in t0
```

```
ld   t1, 116(zero)    # carica var. c in t1
```

```
add  t2, t0, t1        # mette la somma in t2
```

```
sd   t2, 100(zero)    # memorizza t2 in var. a
```



esercizio con array

programma C

```
LONG b, c
```

```
LONG A[20]
```

```
b = A[c]
```

mappatura delle variabili in memoria

B: indirizzi $100_{\text{dec}} \dots 107_{\text{dec}}$

C: indirizzi $108_{\text{dec}} \dots 115_{\text{dec}}$

A[0]: indirizzi $200_{\text{dec}} \dots 207_{\text{dec}}$

A[1]: indirizzi $208_{\text{dec}} \dots 215_{\text{dec}}$

...

A[19]: indirizzi $20 \cdot 19 \dots 20 \cdot 19 + 7$



array

in C la dichiarazione `LONG A[20]` è un *modo compatto* per dichiarare e allocare 20 variabili di tipo intero lungo un colpo solo:

`A[0], A[1], ..., A[19]`

il vantaggio è che possiamo usare un'espressione come indice (variabile parametrica): `A[i]`

`LONG A[20]` può essere mappato in memoria come 20 variabili di tipo intero lungo consecutive:

- `A: .zero 160`
- `A[0] ⇒ 200, ..., 207`
- `A[1] ⇒ 208, ..., 215`
- ...

`&A[i]` indica il primo indirizzo al byte dove è mappato l'intero lungo `A[i]`

l'indirizzo in memoria di `A[i]` è allora

- $200 + 8 \times i$ equivalente a: $200 + \text{sizeof}(\text{LONG}) \times i$
- dove 200 è l'indirizzo di `A[0]`, la base, ovvero `&A[0]`



allineamento in memoria delle variabili

le variabili collocate in memoria sono allineate (cioè poste a indirizzi) secondo il loro tipo:

- *byte* (8 bit) a indirizzo qualunque: 0, 1, 2, 3, 4, ...
- *mezza parola* (*half* – 16 bit) a indirizzo pari: 0, 2, 4, 6, ...
- *parola* (*word* – 32 bit) a indirizzo multiplo di quattro: 0, 4, 8, 12, ...
- *parola doppia* (*dword* – 64 bit) a indirizzo multiplo di otto: 0, 8, 16, 24, ...

si può cambiare l'allineamento con una direttiva

```
.align 0      // nessun allineamento (come per byte)
.align 1      // allinea a indirizzo pari
.align 2      // allinea a indirizzo multiplo di 4
.align 3      // allinea a indirizzo multiplo di 8
```

esempio di allocazione non allineata

```
    .align 1    // allinea a indirizzo pari
VAR: .word    .. // VAR (32 bit) a ind. pari (non mul. 4)
```



direttive fondamentali assembler – i

direttive fondamentali:

.align <i>num</i>	allinea il dato dichiarato di seguito secondo l'argomento <i>num</i> : <i>num</i> = 0,1,2,3 indica byte, mezza parola, parola e parola doppia
.ascii <i>str</i>	riserva spazio per la stringa <i>str</i> nel segmento dati, senza aggiungere il terminatore di stringa
.asciiz <i>str</i>	riserva spazio per la stringa <i>str</i> nel segmento dati e aggiungi il terminatore di stringa (NULL)
.byte <i>b1</i> , ...	riserva spazio nel segmento dati per i byte elencati e inizializzali con i valori a 8 bit <i>b1</i> , ...
.data <i>ind</i>	dichiara il segmento dati con indirizzo iniziale <i>ind</i> (default <i>ind</i> = 0x 0000 0000 1000 0000)
.dword <i>d1</i> , ...	riserva spazio nel segmento dati per le parole doppie elencate e inizializzale con i valori a 64 bit <i>d1</i> , ..., allneandole a indirizzi multipli di otto (.align cambia allineamento)
.eqv <i>sym</i> , <i>val</i>	dichiara un simbolo: il simbolo <i>sym</i> ha valore numerico <i>val</i> , senza allocare memoria (funziona come #define in C ed è usato soprattutto per definire lo spiazzamento in pila) attenzione: il simbolo <i>sym</i> dichiarato è solo locale al modulo



Direttive fondamentali assembler – ii

.globl <i>sym</i>	esporta il simbolo <i>sym</i> verso tutti gli altri moduli oggetto
.half <i>h1</i> , ...	riserva spazio nel segmento dati per le mezze parole elencate e inizializzale con i valori a 16 bit <i>h1</i> , ..., allneandole a indirizzi pari (.align cambia allineamento)
.space <i>num</i>	riserva spazio nel segmento dati per <i>num</i> byte, senza inizializzarli - si usa principalmente per vettore e struct, e in luogo di .dword , .word , .half e .byte se il valore iniziale manca - utilizzata da RARS
.text <i>ind</i>	dichiara il segmento testo (codice) con indirizzo iniziale <i>ind</i> (default <i>ind</i> = 0x 0000 0000 0040 0000)
.word <i>w1</i> , ...	riserva spazio nel segmento dati per le parole elencate e inizializzale con i valori a 32 bit <i>w1</i> , ..., allneandole a indirizzi multipli di quattro (.align cambia allineamento)
.zero <i>num</i>	come .space - usato in genere per lo spazio dati BSS (Block Starting Symbol): porzione del segmento dati statici dove sono allocate le var. statiche non inizializzate (linux e windows iniz. a 0 questi valori) - utilizzata da GCC

altre direttive:

.macro <i>a1</i> , ...	dichiarazione di macro con argomenti <i>a1</i> , ...
.end_macro	fine dichiarazione di macro
.rodata	dichiara segmento di dati imm modificabili (read-only) - GCC



variabili

in RISC V le variabili sono manipolate tramite indirizzo simbolico o nome di registro

occorre distinguere tra diverse classi di variabili:

- **globale o statica**
- **parametro**
- **locale**
- **(dinamica)**

SP → 0000 003f ffff fff0_{esa}

0000 0000 1000 0000_{esa}

PC → 0000 0000 0040 0000_{esa}

0



variabile globale nominale

- la **variabile globale** è collocata in memoria a indirizzo fisso stabilito dall'assemblatore
- per comodità l'indirizzo simbolico della variabile globale coincide con il nome della variabile
 - **solitamente l'indirizzo simbolico è scritto in carattere maiuscolo**, ma solo per comodità di visualizzazione
- gli ordini di dichiarazione e disposizione in memoria delle variabili globali coincidono



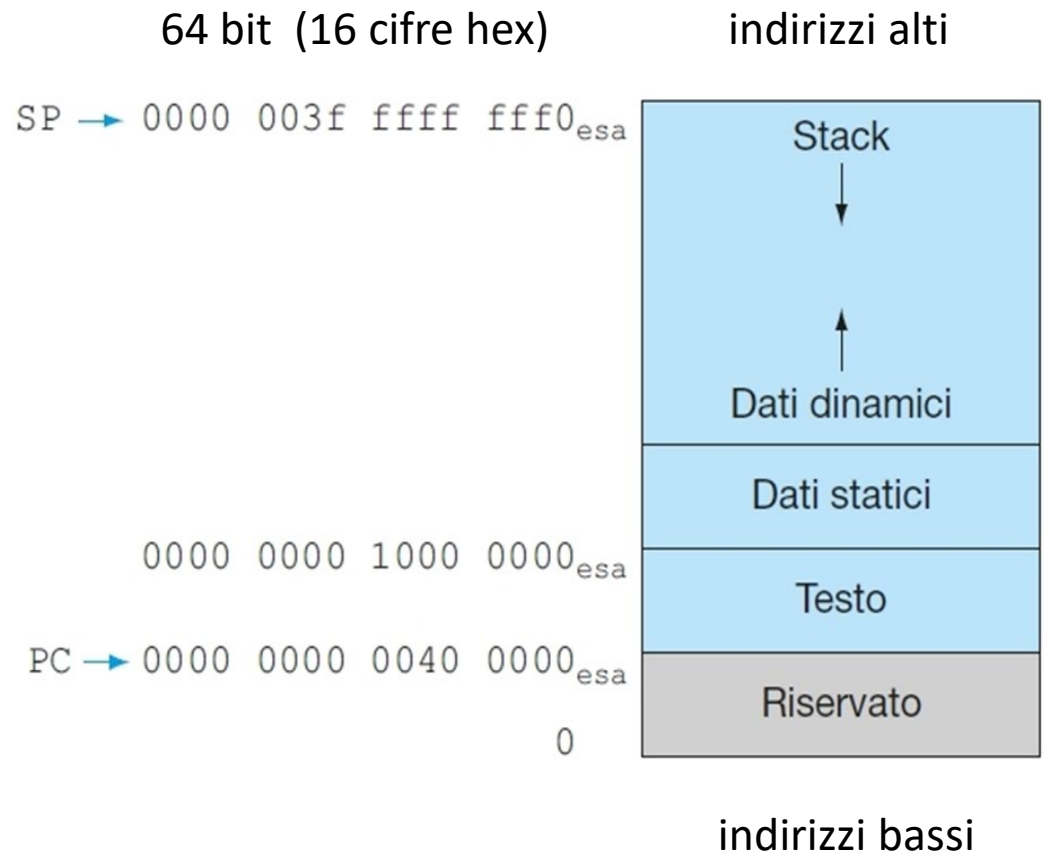
segmento dati statici per variabili globali

le variabili globali sono allocate nel segmento dei dati statici a partire dall'indirizzo:

0x 0000 0000 1000 0000

l'allocazione in memoria delle variabili globali procede in ordine di dichiarazione (come sono elencate nel sorgente C)

alle variabili si assegna spazio dagli indirizzi bassi verso quelli alti



dichiarazione variabili globali – scalare e vettore

```
// LONG 64 bit
char c = 'A'
short int h
int i = 1
LONG d = 2
LONG * p1 = NULL
char * p2
LONG vet [10]
char str [10]
```

```
.data // segmento dati
C: .byte 65 // valiniz ASCII 'A' = 65
H: .half // non inizializzato
I: .word 1 // valore iniziale = 1
D: .dword 2 // valore iniziale = 2
P1: .dword 0 // inizializzato a NULL
P2: .dword // non inizializzato
VET: .zero 80 // 10 elementi × 8 byte
STR: .zero 10 // 10 elementi × 1 byte
```

il valore iniziale è facoltativo

il puntatore (che di fatto è un indirizzo) è sempre una parola doppia da 64 bit, indipendentemente dal tipo di oggetto puntato

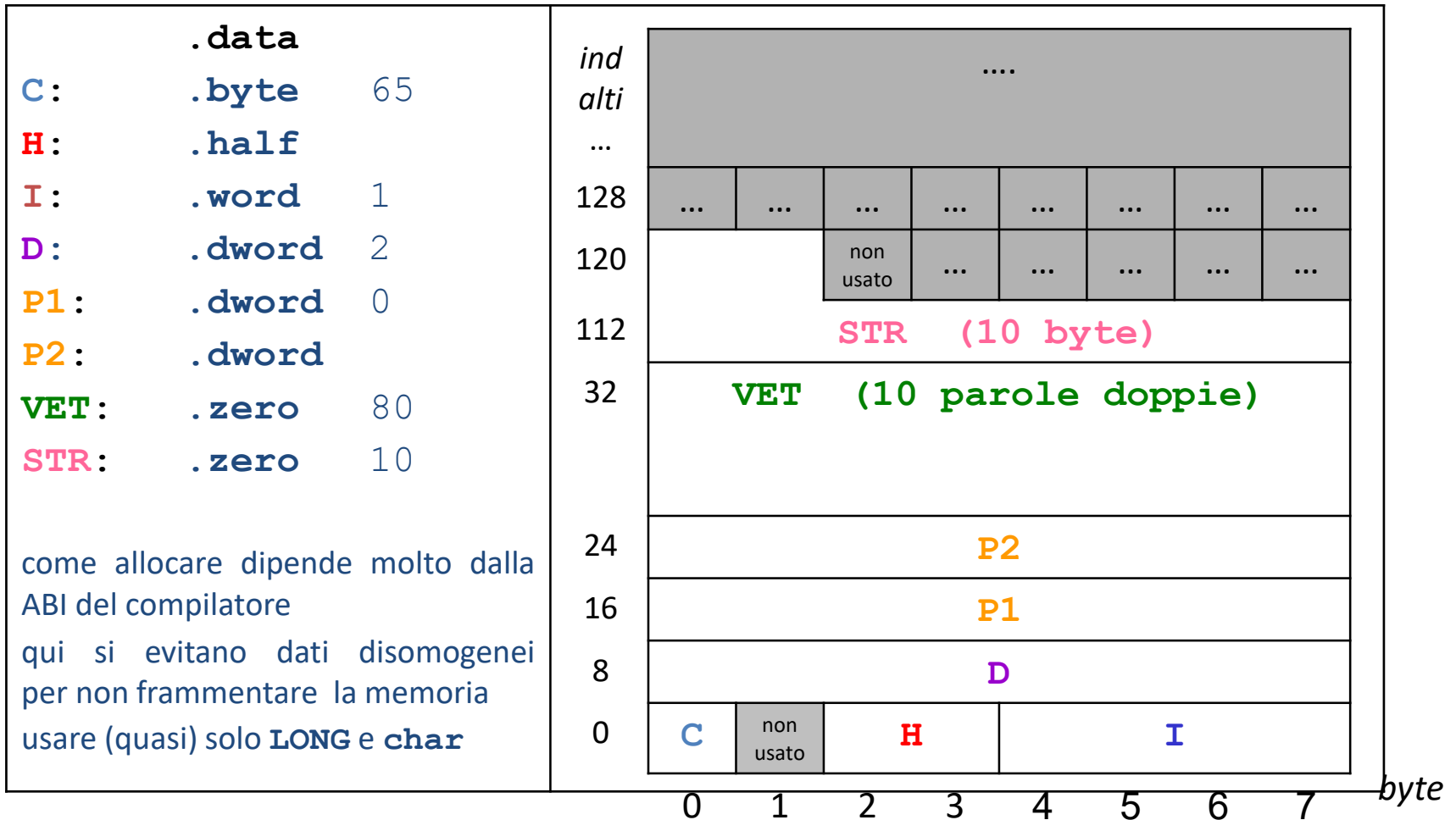
alcuni simulatori RISC-V, p. es. RARS, richiedono che le direttive **.dword**, **.word**, **.half** e **.byte** specifichino **sempre un valore iniziale** per la variabile; è comunque sempre possibile allocare una variabile con **.zero n** (n = ingombro var in byte)

RARS usa **.space n**, equivalente



variabile globale – allocazione allineata

– little endian



variabile globale – struct (per completezza)

```
struct {  
    LONG a  
    int b  
    char c  
} card
```

ind alti				
	c	8 bit (1 byte)	12	sbi
	b	32 bit (4 byte)	8	
	a	64 bit (8 byte)	0	
ind bassi				

```
card.a = 2  
card.b = 2  
card.c = 2
```

```
.data // segmento dati
```

```
NOME_STRUCT: .zero n // riserva n byte
```

il compilatore riserva uno spazio di n byte (dagli indirizzi bassi verso quelli alti) sufficiente a contenere in sequenza i campi della *struct* ci sono varie convenzioni su come calcolare il numero n di byte, se i campi sono allineati o no, o se ogni campo occupa esattamente lo spazio sufficiente a contenerlo, o più spazio in parte libero (*padding*) per queste convenzioni occorre vedere la ABI (Application Binary Interface) specifica usata dal compilatore

si accede a un campo interno della *struct* mediante lo spiazzamento del campo rispetto all'indirizzo iniziale della *struct* (vedi anche array)

```
CARD: .zero 13 // 13 = 8 + 4 + 1 byte  
la t0, CARD // carica ind di CARD in t0  
li t1, 2 // carica costante 2 in t1  
sd t1, 0(t0) // aggiorna campo a di CARD  
sw t1, 8(t0) // aggiorna campo b di CARD  
sb t1, 12(t0) // aggiorna campo c di CARD
```



vettore – accesso costante – vettore di interi LONG

```
// variabili globali
LONG v [5] // vettore
...
// accesso costante
v[0] = 6
...
v[2] = v[1] + 7
...
```

attenzione estremi vettore

```
// riserva 5 × 8 = 40 byte per v
V: .zero 40
// v[0] = 6
li    t0, 6           // carica cost 6
la    t1, v          // carica ind iniz di v
sd    t0, 0(t1)       // memorizza elem v[0]
// v[2] = v[1] + 7
la    t0, v          // carica ind iniz di v
ld    t1, 8(t0)       // carica elem v[1]
li    t2, 7           // carica cost 7
add   t1, t1, t2       // calcola espr v[1] + 7
sd    t1, 16(t0)      // memorizza elem v[2]
```

il compilatore in realtà espanderebbe la pseudo-istruzione
la t1, V con indirizzo V in questo modo:

```
auipc t1, %pcrel_hi(v)
addi  t1, t1, %pcrel_lo(v)
```

tramite **auipc** carica in **t1** i 20 bit più significativi di V sommati all'MSB dei meno significativi di V e poi
tramite **addi** concatena in **t1** i 12 bit meno significativi di V
l'indirizzo V è risolto come spiazzamento rispetto ad **auipc**,
sono possibili ottimizzazioni trattando costanti e aritmetica



vettore – scansione sequenziale con indice – vettore di interi LONG

```
// variabili globali
LONG v [5] // vettore
LONG a      // indice
...
// testata del ciclo
for (a = 0; a <= 4; a++) {
    v[a] = v[a] + 7
} /* end for */
// seguito del ciclo
...
```

attenzione estremi vettore

l'istruzione **slli** con scorrimento 3
allinea l'indice, moltiplicando
per $2^3 = 8 = \text{sizeof}(\text{LONG})$

```
V:    .zero 40           // riserva 40 byte per v
A:    .dword             // parola doppia per a
      // esegui a = 0 come già visto
FOR: la  t0, A           // carica ind di a
      ld  t1, (t0)        // carica a
      li  t2, 4           // carica cost 4
      bgt t1, t2, END     // se a > 4 va' a END
      la  t0, v          // carica ind iniz di v
      la  t1, A          // carica ind di a
      ld  t2, (t1)        // carica a
      slli t2, t2, 3       // allinea indice
      add t0, t0, t2      // calcola ind di v[a]
      ld  t3, (t0)        // carica elem v[a]
      li  t4, 7           // carica cost 7
      add t3, t3, t4      // calcola espr v[a] + 7
      sd  t3, (t0)        // memorizza elem v[a]
      // esegui a++ come già visto
      j    FOR            // torna a FOR
END: ...                 // seguito ciclo
```

ottimizzazioni possibili trattando costanti e aritmetica



vettore – scansione sequenziale con puntatore – vettore di interi LONG

```
// variabili globali
LONG v [5] // vettore
LONG a      // contatore
LONG * p    // puntatore
...
// testata del ciclo
p = v // p punta a v[0]
for (a = 0; a <= 4; a++) {
    *p = *p + 7
    p++
} /* end for */
// seguito del ciclo
...
attenzione estremi vettore
l'istruzione addi ha incremento 8
= sizeof(LONG)
```

```
V: .zero 40 // riserva 40 byte per v
A: .dword // parola doppia per a
P: .dword // parola doppia per p
la t0, v // carica ind iniz di v
la t1, p // carica ind di p
sd t0, (t1) // memorizza p = v
// esegui a = 0 come già visto
FOR: // verifica a <= 4 come già visto
// *p = *p + 7
la t0, p // carica ind di p
ld t1, (t0) // carica p
ld t2, (t1) // carica oggetto *p
li t3, 7 // carica cost 7
add t2, t2, t3 // calcola espr *p + 7
sd t2, (t1) // memorizza oggetto *p
// p++
la t0, p // (ri)carica ind di p
ld t1, (t0) // (ri)carica p
addi t1, t1, 8 // calcola p++
sd t1, (t0) // memorizza p
// esegui a++ come già visto
j FOR // torna a FOR
END: ... // seguito ciclo
```

ottimizzazioni possibili trattando costanti e aritmetica



parametri in ingresso alla funzione

i primi sei parametri vanno passati nei registri *a2* (primo nella testata della funzione), *a3*, *a4*, *a5*, *a6* e *a7* (sesto)

- se sono di tipo scalare o puntatore (parole a 64 bit)
 - il nome di vettore è gestito come il valore di un puntatore al primo elemento (che ha indice 0)
- per passare una *struct* si veda la ABI del compilatore
 - la ABI di *gcc* impone di passare l'indirizzo del primo campo della *struct*

gli eventuali parametri rimanenti vanno impilati a cura del chiamante, sempre come parole a 64 bit

- è raro che una funzione abbia più di sei parametri

Prima di chiamare la funzione, il chiamante salva sulla pila i registri *t0* – *t6* e *a0* – *a7* (in questo ordine) che il chiamante necessita di riavere inalterati al rientro **Chiamati Caller-Saved**



valore in uscita dalla funzione

il valore in uscita (a 64 bit) va restituito nel registro *a0*

- se è di tipo scalare o puntatore (parola a 64 bit)
- per restituire una *struct* la ABI di *gcc* impone di restituire l'indirizzo del primo campo della *struct*



variabile locale nominale

la variabile locale può essere *gestita in vari modi*, secondo il tipo di variabile e il grado di ottimizzazione del codice, e anche in dipendenza di come la variabile viene utilizzata

- **variabile di tipo scalare o puntatore**
 - viene associata/copiata in un registro del blocco $s0-s11$ *, *se per altro motivo non deve avere un indirizzo di memoria – vedi precisazioni sotto*
 - altrimenti nell'area di attivazione della funzione
 - **variabile di tipo scalare, che viene acceduta (anche) tramite un puntatore** (*cioè nel codice viene indicato l'indirizzo della variabile*)
 - viene associata/copiata nell'area di attivazione della funzione, perché deve avere un indirizzo!! **Salvata nel RdA**
 - variabile di tipo *array* (o *struct*)
 - viene associata/copiata sempre nell'area di attivazione della funzione
- * se il registro $s0$ (chiamato anche $x8$) è usato come *frame pointer* (*fp*), si inizia da $s1$





POLITECNICO
MILANO 1863

sottoprogramma – (procedura / funzione)

sottoprogramma (procedura / funzione) – i

in linguaggio di alto livello, ad esempio in C, la **chiamata a sottoprogramma** ha come effetto la creazione di un'**area di attivazione** (*activation record* o *frame*) sulla **pila** (*stack*)

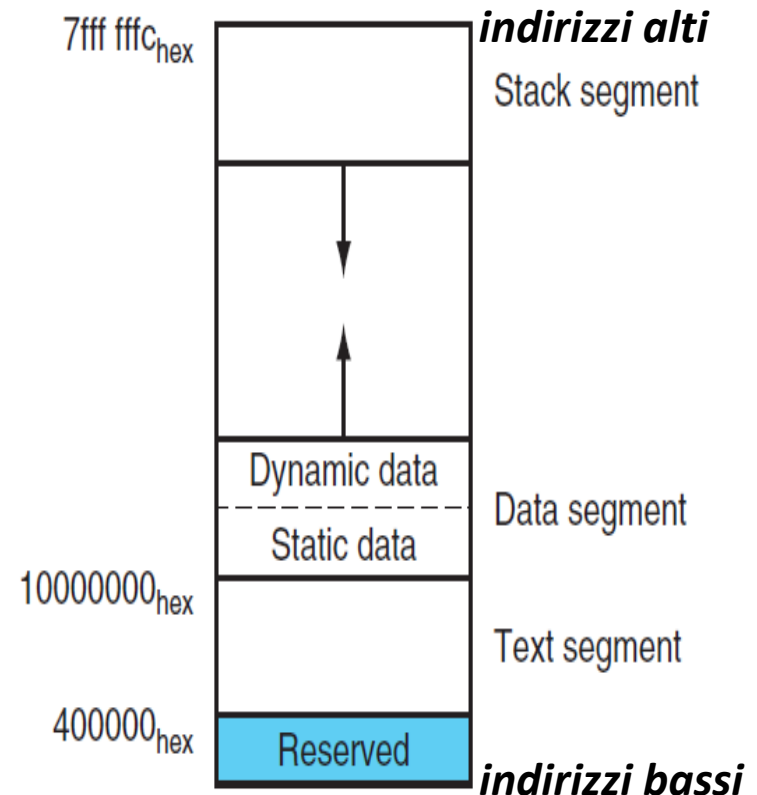
- a ogni chiamata (attivazione) viene creata un'area
- quando il sottoprogramma termina l'area viene rilasciata (o *deallocata*) dalla pila
- con sottoprogrammi annidati le aree vengono impilate sulla pila e l'ultima area corrisponde al sottoprogramma correntemente in esecuzione
- l'area di attivazione di *main* è la prima ad essere creata quando il programma viene lanciato, e l'ultima ad essere rilasciata (deallocata)



sottoprogramma – ii

l'area di attivazione è associata in modo opportuno alle seguenti informazioni

1. i **parametri** formali (e loro valori)
2. l'**indirizzo di rientro** (ritorno) al chiamante
3. le **informazioni per gestire lo spazio allocato** per l'area di attivazione
4. le **variabili locali**
5. il **valore restituito**



sottoprogramma – iii

il programma **chiamante** deve eseguire le operazioni seguenti

- predisporre i parametri di ingresso della procedura in un posto accessibile alla procedura (**registri a2-a7**)
- trasferire il controllo alla procedura / funzione (**jal**)

la procedura / funzione **chiamata** deve eseguire le operazioni seguenti

- allocare lo spazio di memoria necessario sia per i suoi dati, sia per sua esecuzione (**area di attivazione**)
- eseguire il compito richiesto
- memorizzare il risultato in un luogo accessibile al chiamante (**registri a0 e a1**)
- restituire il controllo al chiamante (**ret**)



sottoprogramma – iv

per *chiamare* una procedura / funzione, abbiamo bisogno di dare il controllo al codice della procedura / funzione stessa; dobbiamo dunque saltare alla prima istruzione del codice della procedura / funzione

abbiamo già visto istruzioni per saltare e, in particolare, l'istruzione **jal** (jump & link) (o una psuedoistruzione equivalente)

jal ra, indirizzo #inLinguaggioMacchina_spi20

salta all'indirizzo, e mette nel registro **ra** il valore del PC dell'istruzione successiva alla chiamata **jal** (ossia PC + 4)

quindi se la procedura / funzione A vuole chiamare B . . . e la prima istruzione di B è memorizzata all'indirizzo `indirizzodiB`

– A può usare: **jal ra, indirizzodiB**

una volta saltati nel codice di B (da A), al termine di B il controllo va restituito ad A per tornare a eseguire la parte rimanente di A

– B può usare: **jr ra** oppure **ret**



riassumendo

il (sotto)programma **chiamante** deve

- mettere i valori dei parametri da passare alla procedura / funzione nei registri **a2**, ..., **a7**
- utilizzare l'istruzione **jal ra, add** per saltare alla procedura / funzione e salvare il valore dell'indirizzo della istruzione successiva (**PC + 4**) nel registro **ra**

il sottoprogramma (procedura / funzione) **chiamato** deve

- eseguire il compito richiesto
- memorizzare il risultato nei registri **a0**, **a1**
- restituire il controllo al chiamante con l'istruzione **jr ra**



effetto collaterale (*side effect*)

- le procedure / funzioni possono avere bisogno di utilizzare dei registri per le proprie operazioni
- questi registri potrebbero contenere dei valori che il chiamante dovrebbe utilizzare successivamente e che verrebbero modificati

side effect

- è necessario fare in modo che
 - **i registri importanti** per il prosieguo della computazione del chiamante (e.g., il main) **vengano salvati in un'area di memoria** prima dell'esecuzione delle istruzioni della procedura/funzione chiamata
 - poi venga eseguita la procedura / funzione chiamata
 - e infine **i registri salvati vengono ripristinati dalla memoria** per continuare con le istruzioni del chiamante (i.e., main)



convenzioni per il salvataggio dei registri

- l'esecuzione di un sottoprogramma (*callee*) non deve interferire con l'ambiente del (sotto)programma chiamante (*caller*)
- Dei valori nei registri usati dal chiamato (*callee*) va fatta una copia, il chiamato (*callee*) può poi usarli liberamente e alla fine ri-aggiornarli con i valori che avevano in origine.

convenzioni adottate in **RISC V**

- per ottimizzare gli accessi alla memoria, il chiamante e il chiamato salvano **sulla pila** soltanto un particolare gruppo di registri ciascuno
- il chiamato può usare la pila anche per le strutture dati locali (p.es. array e struct) e per le variabili locali

<i>preservato dal chiamato (registri callee-saved)</i>	<i>non preservato dal chiamato (registri caller-saved)</i>
registro frame pointer: <i>fp</i>	registri temporanei: <i>t0-t6</i>
registro return address: <i>ra</i>	registri argomento: <i>a2-a7</i>
registri saved: <i>s0-s11</i>	registri di ritorno: <i>a0-a1</i>

t-a in
utilizzo
posizionati
nella Stack
dal Chiamante

Se il callee non è un programma foglia, viene salvato *ra* sullo stack



annidamento di procedure / funzioni

MAIN:

...
jal ra, A
...

A:

...
jal ra, B
...
...
jr ra

B:

...
jal ra, C
...
...
jr ra

C:

...
...
...
...
jr ra

Codice del
main

Codice del
sottoprogramma A

Codice del
sottoprogramma B

Codice del
sottoprogramma C

- ogni volta che invochiamo *jal*, il registro *ra* viene riscritto con l'indirizzo dell'istruzione di rientro, pertanto se le procedure / funzioni sono annidate si perde il valore precedente!

Nel caso di una procedura/funzione che NON è foglia,

è

necessario salvare (nel record di attivazione della procedura/funzione chiamata) il valore di *ra* subito dopo l'esecuzione di *jal*



problemi

- mantenere i valori passati come parametri alla procedura / funzione
- una procedura / funzione può avere bisogno di più registri rispetto ai sei a disposizione per i parametri e ai due per la restituzione dei valori
- salvare i registri che una procedura / funzione potrebbe modificare, ma che il (sotto)programma chiamante ha bisogno di mantenere inalterati
- fornire lo spazio necessario per le variabili locali alla procedura / funzione
- gestione di procedure / funzioni annidate (che richiamano al loro interno altre procedure / funzioni) e procedure / funzioni ricorsive (che invocano sé stesse)

soluzione



utilizzo della pila (stack)



pila (*stack*)

- la pila (*stack*) è **una struttura dati** costituita da una coda LIFO (*last-in-first-out*)
- i dati vengono inseriti nella pila con l'operazione **push**
- i dati vengono estratti dalla pila con l'operazione **pop**
- è necessario un puntatore alla **cima** (*top*) della pila per salvare i registri che servono al sottoprogramma chiamato
- il registro **sp** (*stack pointer* o *puntatore alla pila*) contiene l'indirizzo della cima della pila e viene aggiornato ogni volta che viene inserito o estratto il valore di un registro



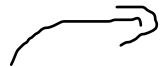
gestione della pila in RISC V

- la pila cresce da indirizzi di memoria alti verso indirizzi bassi
- il registro **sp** contiene l'indirizzo della cella **occupata** in cima alla pila
- l'inserimento di un dato nella pila (operazione di **push**) avviene
 - **decrementando** **sp** per allocare lo spazio -- aumentando quindi la dimensione della pila
 - poi eseguendo l'istruzione **sd** per inserire (scrivere) il dato
- l'estrazione di un dato dalla pila (operazione di **pop**) avviene
 - eseguendo l'istruzione **ld** per estrarre (leggere) il dato
 - poi **incrementando** **sp** per rilasciare lo spazio (e dunque per eliminare il dato) -- riducendo quindi la dimensione della pila



operazioni sulla pila

PUSH (registro) mette in cima alla pila il valore contenuto nel registro

 Stack Pointer

`addi sp, sp, -8` # fa spazio sulla pila

`sd registro, 0(sp)` # mette il registro nella pila

POP (registro) toglie l'elemento in cima alla pila e lo mette nel registro

`ld registro, 0(sp)` # copia il primo elemento della pila nel registro

`addi sp, sp, 8` # libera la cima (modifica il valore di `sp` che punta alla prima cella libera)

- la pila/stack permette di allocare uno spazio di memoria privato (**FRAME**) per ogni istanza / chiamata di procedura
 - Per es. ci saranno più istanze attive nel caso di ricorsione
- la pila è dinamica
 - la dimensione si adatta alle necessità



gestione della pila in RISC V – i

tutto lo spazio in pila di cui una procedura / funzione ha bisogno (**area di attivazione – stack frame**) viene *esplicitamente* allocato **dal programmatore** in una sola volta, come prima parte del codice di una procedura / funzione

lo spazio nella pila viene allocato **sottraendo** al registro *sp* il numero di byte necessari; per esempio:

```
addi sp, sp, -24    # alloca 24 byte nella pila
```



gestione della pila in RISC V – ii

quando una procedura / funzione rientra, essa rilascia (dealloca) l'area di attivazione incrementando il registro **sp** della stessa quantità di cui lo aveva decrementato alla chiamata; per esempio

```
addi sp, sp, 24      # dealloca 24 byte nella pila
```

è necessario rilasciare lo spazio allocato per evitare di riempire tutta la memoria

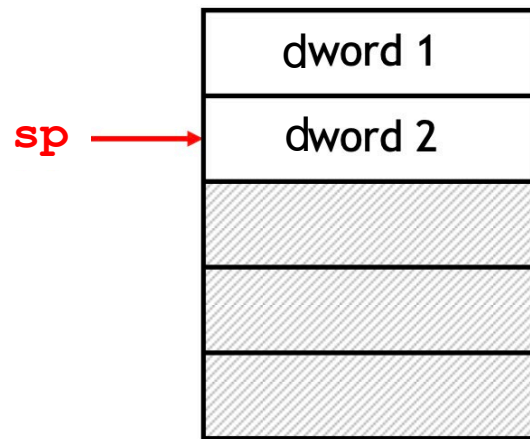


gestione della pila in RISC V – iii

inserire elementi nella pila (PUSH)

```
addi sp, sp, -16  
sd    t1, 8(sp)  
sd    t2, 0(sp)
```

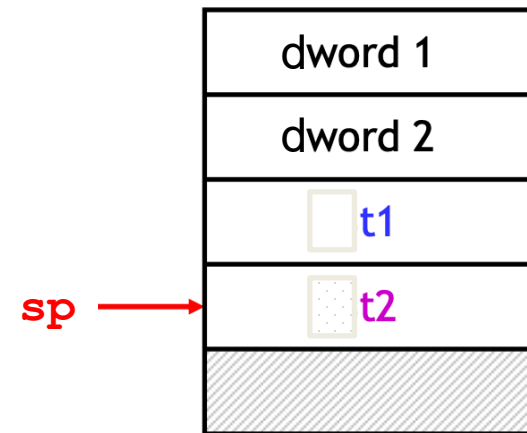
indirizzi alti



Before

*crescita
pila*

indirizzi bassi



After

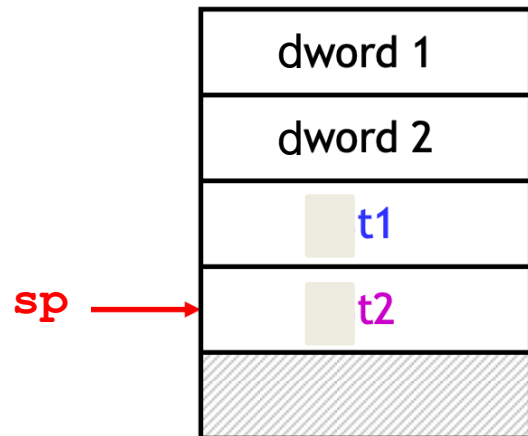


gestione della pila in RISC V – iv

estrarre elementi dalla pila (POP)

```
ld    t0, 0(sp)  
addi  sp, sp, 8
```

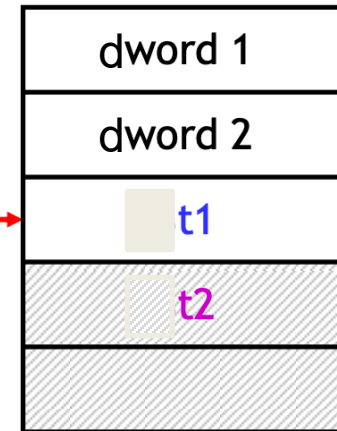
indirizzi alti



Before

*crescita
pila*

indirizzi bassi



After

la pila

quando si chiama una procedura / funzione i registri utilizzati dal chiamato vanno

- salvati nella pila
- il loro contenuto va ripristinato dopo l'esecuzione del codice che consente la chiamata della procedura / funzione



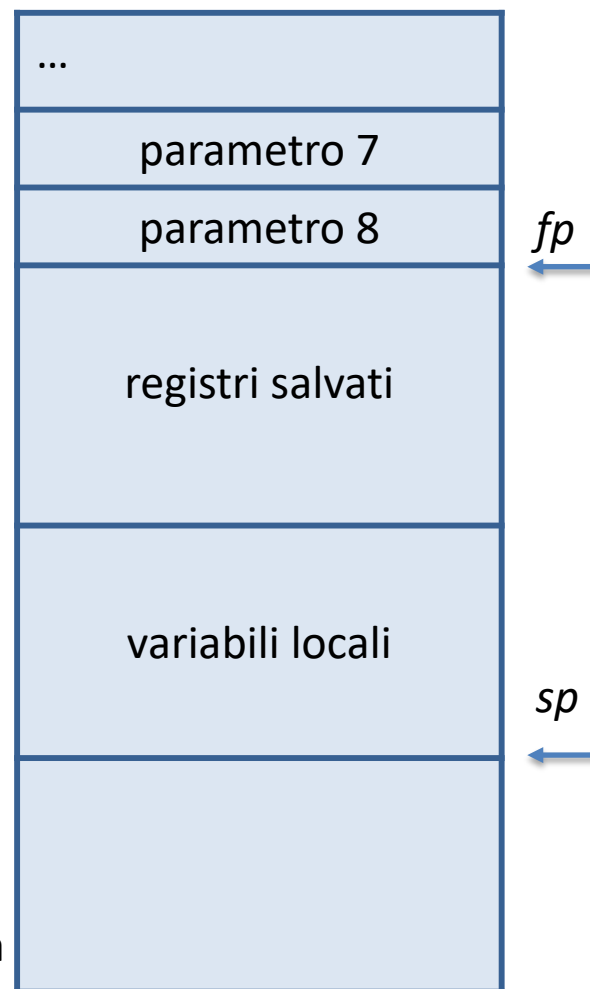
area di attivazione – *activation area* o *frame*

- L'area di attivazione è la parte di memoria allocata per una procedura / funzione per
 - salvare copia dei registri usati dal sottoprogramma
 - allocare memoria per le variabili locali definite dal sottoprogramma
- Il registro *fp* punta alla prima parola dell'area
- Il registro *sp* punta all'ultima parola dell'area
- può contenere i parametri eccedenti (i primi sei stanno nei registri *a2-a7*)

indirizzi alti di memoria

crescita
stack

indirizzi bassi di memoria



Si parla di “Record di Attivazione” (RdA) per fare riferimento
all'area di memoria occupata con informazioni salvate dal chiamante
(e.g., parametri) + la sua area di attivazione (cioé informazioni salvate dal chiamato)



registro *frame pointer*

a ogni operazione di PUSH / POP modifichiamo sempre dinamicamente *sp*, ma

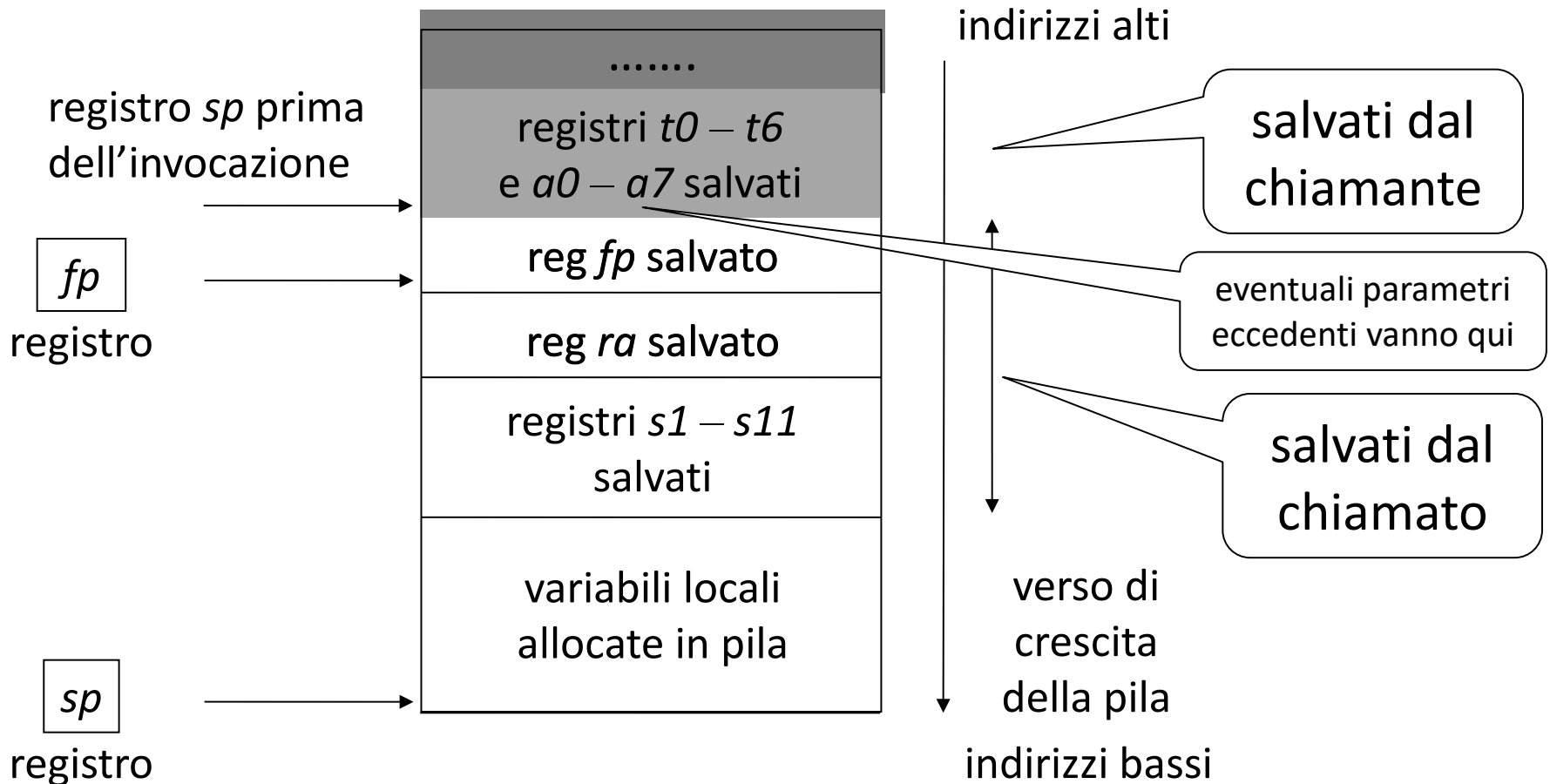
- può essere utile sfruttare anche il registro *fp* (*frame pointer*) per *ricordare* l'inizio dell'area
- questo serve a cancellare velocemente l'area di attivazione al rientro dalla procedura / funzione

l'uso del registro *fp* è del tutto opzionale (GCC lo usa)

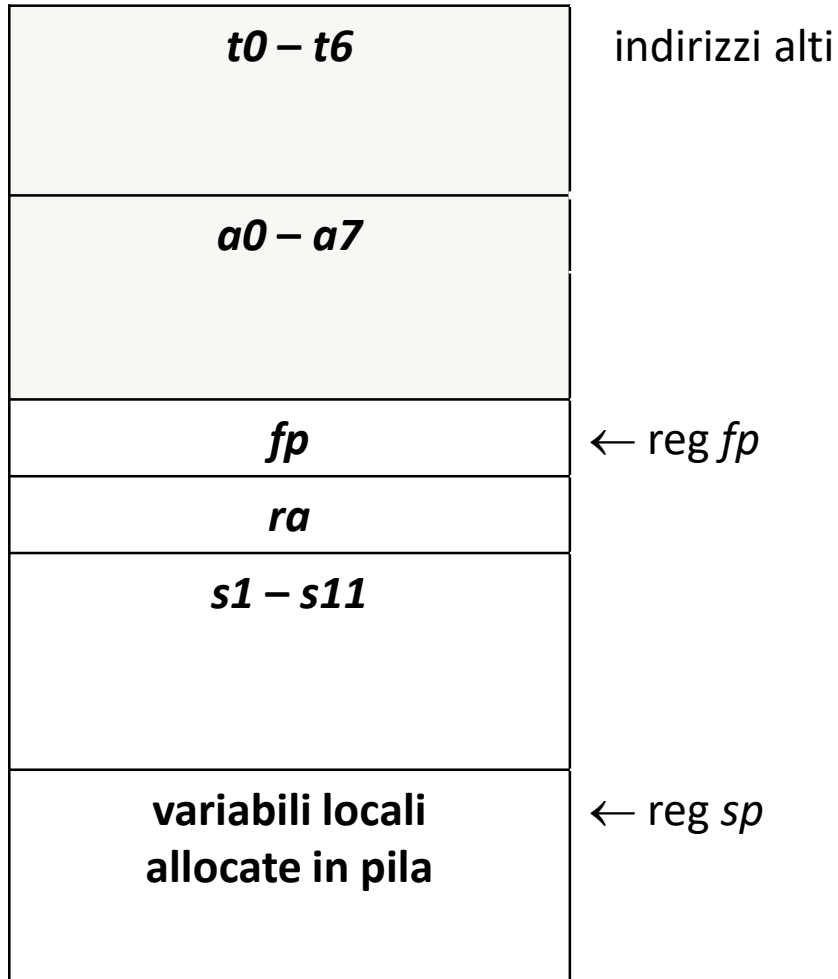
- **il programmatore deve assegnare il valore a *fp* all'inizio della procedura / funzione (dopo averne salvato il valore nella pila)**
- il registro *fp* permette di *concatenare* le aree di attivazione impilate, come in una lista con puntatori



area di attivazione – forma generale



area di attivazione – in dettaglio



- ❑ la pila è un vettore (array) di parole doppie (cioè parole da 64 bit)
- ❑ la cima della pila è sempre allineata a parola doppia
- ❑ i registri vengono salvati come parole doppie, a prescindere dai bit in uso (*byte, half, word, dword*)
- ❑ lo spazio per le variabili locali in pila è un blocco di parole doppie
- ❑ le variabili locali in pila occupano tale spazio secondo il rispettivo ingombro e allineamento
 - lo spazio può essere frammentato
 - sono dettagli dipendenti dalla ABI
 - qui tali dettagli non sono necessari



precisazioni – salvare i registri

i registri $t0 - t6$ e $a0 - a7$ vanno salvati e poi ripristinati da parte del **chiamante (caller)**, se **esso** deve o preferisce poterli riavere inalterati dopo la chiamata

- salva solo quelli che deve o preferisce riavere inalterati
- se non occorre salvarne nessuno, essi non hanno spazio riservato in pila, e non vanno né salvati né ripristinati

i registri $s1 - s11$ vanno salvati e poi ripristinati da parte del **chiamato (callee)**, se **esso** li usa per allocare variabili locali

- salva solo quelli che vengono assegnati a variabili locali
- se non occorre salvarne nessuno, essi non hanno spazio riservato nell'area di attivazione, e non vanno né salvati né ripristinati



precisazioni – salvare i registri

il registro *ra* (*return address*) è facoltativo
va usato se la procedura / funzione ne chiama un'altra, o anche
se chiama sé stessa (è *ricorsiva*)

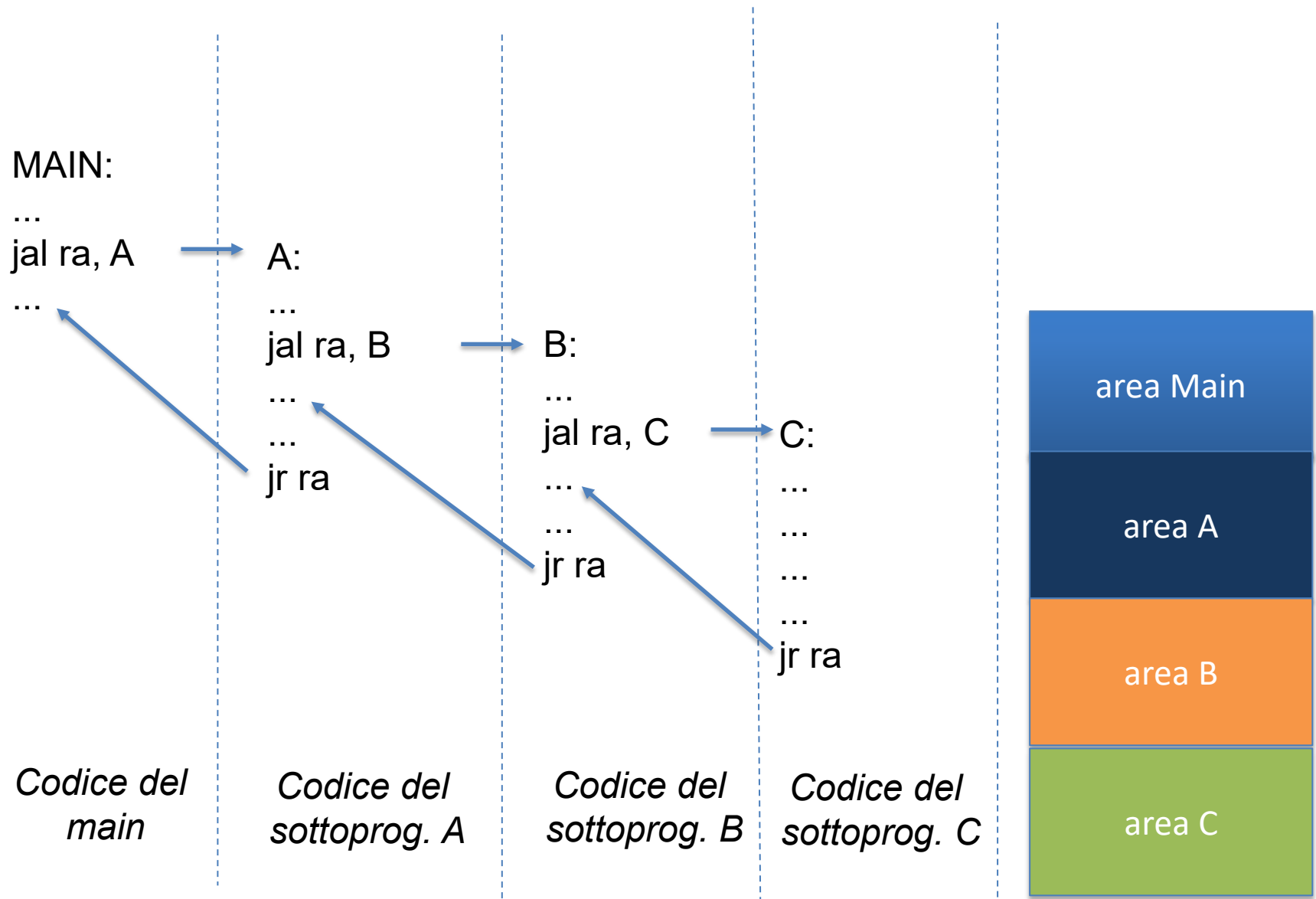
- viene salvato da parte del **chiamato**

non va usato se la procedura / funzione non ne chiama nessuna
altra e neppure sé stessa (procedura / funzione *foglia*)

- **in questo caso il registro *ra* non ha spazio riservato nell'area di attivazione, e non va né salvato né ripristinato**



allocazione delle aree in caso di procedure annidate



registri salvati dal chiamante

t0, ..., t6 (*caller-saved registers*)

- registri usati per valori temporanei
- poiché il *chiamato* potrebbe modificare questi registri, se il *chiamante* ha proprio la necessità di preservarne qualcuno, sarà compito del *chiamante* salvarli !!

a2, ..., a7 (*caller-saved registers*)

- registri usati per passare i primi sei parametri
- il *chiamante* assume che il *chiamato* li possa modificare, ad esempio per invocare un'altra procedura
- il *chiamante* li salva solo se ha necessità di preservarli

a0, a1 (*caller-saved registers*)

- registri usati per restituire i risultati
- il *chiamante* assume che il *chiamato* li possa modificare
- il *chiamante* li salva solo se ha necessità di preservarli



minimo salvataggio dei registri

quando la procedura / funzione *chiamata* non deve salvare nulla?

quando la procedura / funzione *chiamante* non deve salvare nulla?



minimo salvataggio dei registri

la procedura / funzione **chiamata** non deve salvare nulla se

- **non** chiama nessun'altra procedura / funzione
(e dunque non modifica *ra*)

e se scrive solo

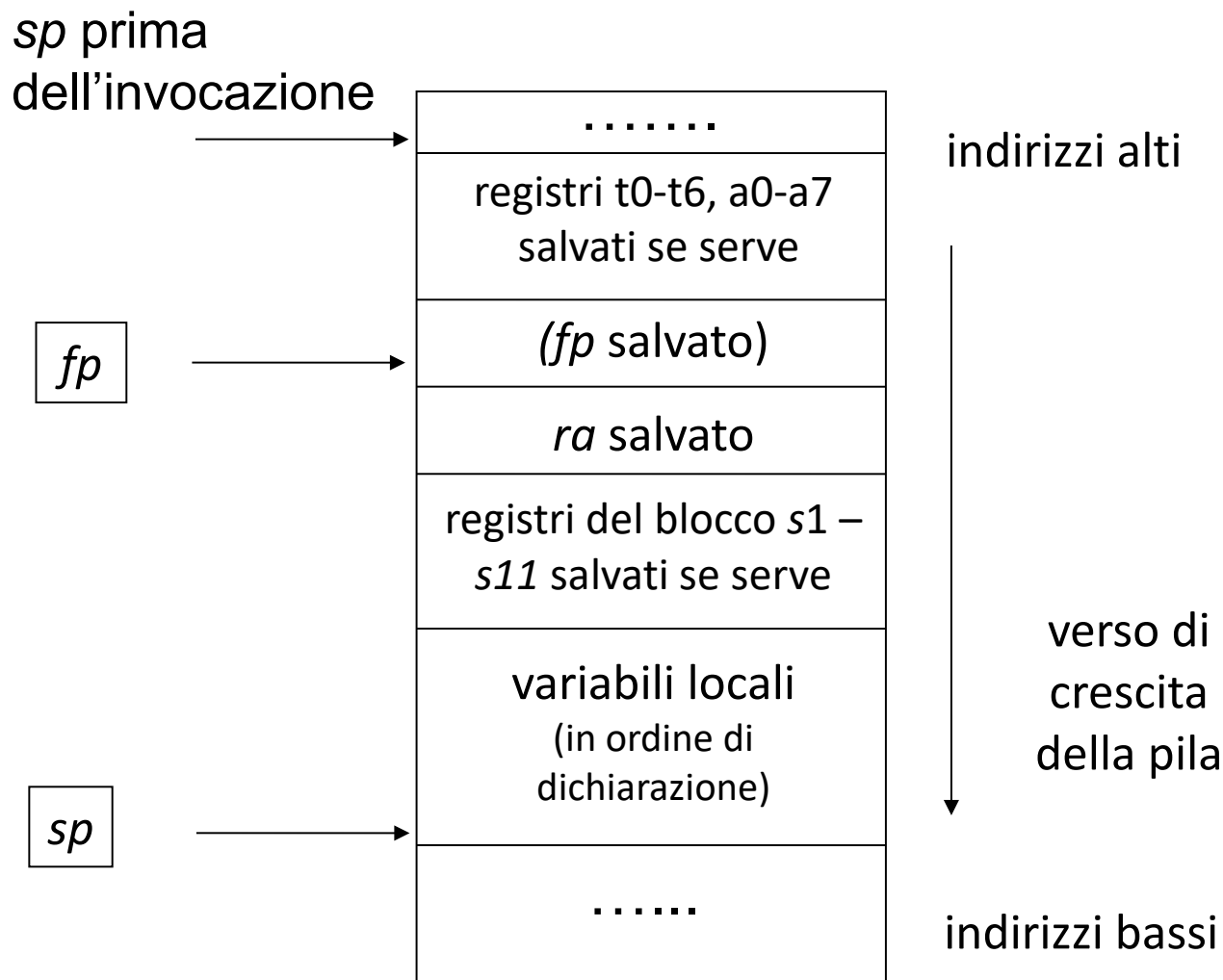
- nei registri temporanei $t0, \dots, t6$
- nei registri dei parametri $a2, \dots, a7$
- in quelli usati convenzionalmente per i risultati $a0, a1$

la funzione **chiamante** non deve salvare nulla se **non** necessita
che certi registri vengano preservati dopo la chiamata

$t0, \dots, t6, a0, a1, a2, \dots, a7$



area di attivazione – forma generale



precisazioni – *frame pointer*

- il registro *fp* (frame pointer) è facoltativo
- il registro *fp* va usato **se serve un riferimento stabile all'area di attivazione della funzione in esecuzione**
 - viene salvato da parte del **chiamato**
- se il registro *fp* non è usato, non ha spazio riservato nell'area di attivazione, e non va né salvato né ripristinato
 - in questo caso la procedura / funzione usa il registro *sp* per fare riferimento al contenuto dell'area di attivazione
 - di fatto il registro *fp* è usato raramente, ma ha una funzione specifica e conviene ricordarne il modo di uso



esempio – i

funzione **chiamata**

```
LONG somma_algebrica (LONG g, LONG h, LONG i, LONG j) {  
    LONG f  
  
    f = (g + h) - (i + j)  
  
    return f  
} /* somma_algebrica */
```



esempio – ii

```
LONG somma_algebrica (LONG g, LONG h, LONG i, LONG j)
{
    LONG f
    f = (g + h) - (i + j)
    return f
} /* somma_algebrica */
```

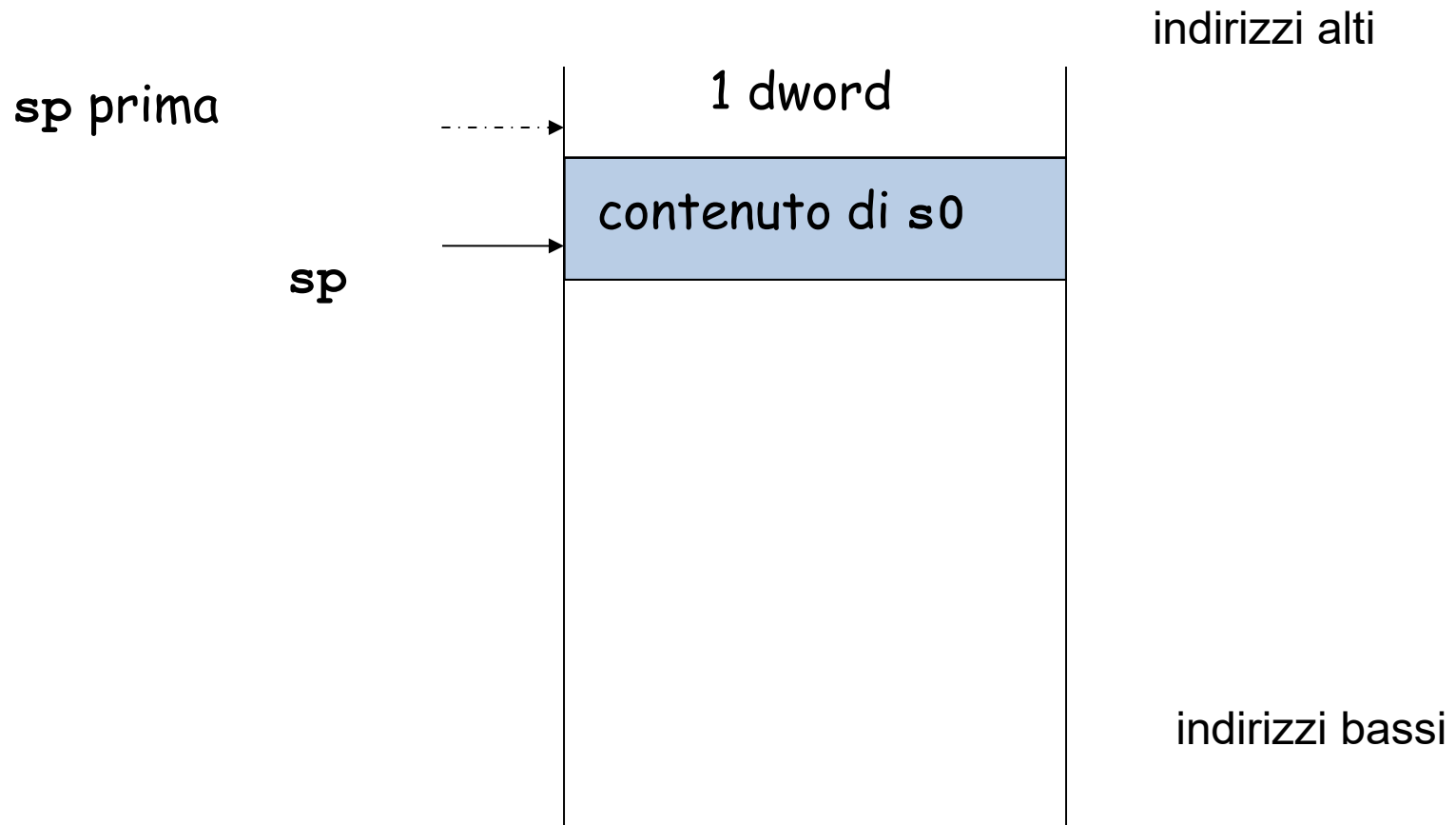
parametri g, h, i e j associati a reg. a2, ..., a5
variabile locale f associata ad s0
il calcolo di f richiede tre registri: s0, t0, t1
necessario salvare SOLO il registro s0 nella pila

example:

```
addi sp, sp, -8      # alloca nella pila
                     # lo spazio per il registro
sd    s0, 0(sp)      # salvataggio di s0
```



esempio – iii



esempio – iv

```
LONG somma_algebrica (LONG g, LONG h, LONG i, LONG j)
{
    LONG f
    f = (g + h) - (i + j)
    return f
} /* somma_algebrica */
```

```
add t0, a2, a3 # t0 ← g + h
add t1, a4, a5 # t1 ← i + j
sub s0, t0, t1 # f ← t0 - t1
```

operazioni previste in assembly per lo statement return del C

```
add a0, s0, zero      # restituisce f copiandolo
                      # nel reg. di ritorno a0
```

alternativa *addi a0, s0, 0* oppure pseudoistruzione *mv a0, s0*

```
# ripristino del vecchio contenuto dei registri
# estraendoli dallo stack
```

```
ld      s0, 0(sp) # ripristino di s0
addi    sp, sp, 8 # deallocazione della pila
          # per eliminare il registro
```

```
jr ra          # rientro al prog. chiamante
```



procedura / funzione foglia – i

- una procedura / funzione **foglia non** presenta al suo interno chiamate ad altre procedure / funzioni (o a sé stessa)
 - **non serve che essa salvi il registro ra** (dato che nessun'altra lo modifica)
- nel caso di procedura / funzione foglia, il **chiamante** salva in pila:
 - i *registri temporanei* e i *registri argomento* di cui avrà bisogno dopo la chiamata $t0-t6$, $a0-a7$, ...
 - eventuali argomenti aggiuntivi



procedura foglia – ii

- nel caso di procedura / funzione foglia, il **chiamato** alloca nella pila:
 - eventuale registro fp
 - i registri non temporanei che deve utilizzare: **s0–s11**
 - strutture dati locali (p.es.: array e matrici) e variabili locali che non stanno nei registri
- il puntatore alla pila (registro **sp**) viene aggiornato secondo il numero di registri memorizzati nella pila
- alla fine della procedura / funzione i registri vengono ripristinati e il puntatore alla pila viene riaggiornato



riassumendo – area di attivazione

- una procedura / funzione viene eseguita in uno spazio *privato* detto **area di attivazione** (*activation record o frame*)
 - area di memoria dove vengono allocate le variabili locali della procedura / funzione
- il programmatore assembler deve provvedere in modo esplicito ad allocare / rilasciare lo spazio necessario
- quando sono permesse chiamate di procedura / funzione annidate, le aree di attivazione vengono allocate e rilasciate come gli elementi di una pila



convenzioni di chiamata – chiamante (1)

prologo del chiamante

- se il chiamante intende usare (quando il chiamato sarà rientrato e quindi dopo l'istruzione di chiamata) i registri $t0-t6$, o alcuni di essi, il chiamante li deve salvare esplicitamente nella pila e poi ripristinarli dopo il rientro del chiamato
- allo stesso modo, se occorre preservarli, il chiamante salva sulla pila gli argomenti $a2-a7$ e il valore dei registri $a0 - a1$
- il chiamante scrive in $a2-a7$ i paramenti in ingresso al chiamato
 - trascuriamo il caso di più di sei parametri

salto a chiamato

- *jal* FUNZ



convenzioni di chiamata – chiamato (1)

prologo del chiamato

- crea area di attivazione: ***addi sp, sp, –dim. area in byte***
- se il registro *fp* è in uso
 - viene salvato sulla pila (*fp* prec)
 - viene aggiornato così da puntare alla cima dell'area di attivazione, cioè alla parola di pila che contiene il valore appena salvato
- se il chiamato non è foglia
 - il registro *ra* viene salvato sulla pila perché sarà usato (e quindi sovrascritto) in chiamata annidata
- salva sulla pila i registri *s0 – s11* assegnati a variabili locali

parte di esecuzione del chiamato



convenzioni di chiamata – chiamato (2)

epilogo del chiamato

- scrive nel registro *a0* il valore di uscita
- ripristina dalla pila i registri *s0–s11* assegnati a variabili locali
- se il chiamato non è foglia, ripristina dalla pila il registro *ra*
- se il registro *fp* è in uso, si ripristina dalla pila
- elimina area di attivazione: ***addi sp, sp, dim. area in byte***

rientro a chiamante

- *jr ra* o *ret*



convenzioni di chiamata – chiamante (2)

epilogo del chiamante

- ripristina dalla pila i registri $a0$ – $a7$ che erano stati preservati per il rientro del chiamato
- ripristina dalla pila i registri $t0$ – $t6$ che con valori inalterati dopo il rientro del chiamato
- trova nel registro $a0$ il valore di uscita dal chiamato



esempio – calcolo del fattoriale in C

definizione: $n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$ con $n \geq 1$ e $0! = 1$

calcoliamo tramite la regola ricorsiva: $n! = n \times (n - 1)!$ con $n \geq 0$

```
LONG fatt (LONG n) {    /* funzione fattoriale      */
    LONG tmp            /* variabile locale di fact */
    tmp = n
    if (tmp == 0) {
        return 1
    } else {
        tmp = tmp * fatt (tmp - 1)
        return tmp
    } /* if */
} /* fatt */

LONG ret = 0            /* variabile globale      */

main ( ) {             /* programma principale   */
    LONG val            /* variabile locale di main */
    val = 4
    ret = fatt (val)
} /* main */
```



traduzione del fattoriale – i

```
.text                                # segmento testo
FATT:  addi sp, sp, -16               # crea area di 16 byte (2 dword)
      sd   ra, 8(sp)                 # salva reg. ra (callee-saved)
      sd   s0, 0(sp)                 # salva reg. s0 (callee-saved)
      mv   s0, a2                    # assegnamento tmp = n
IF:    bne s0, zero, ELSE            # if (tmp != 0) vai a ELSE
THEN:  li  a0, 1                     # prepara valore di uscita
      j    ENDIF                    # vai a ENDIF
ELSE:  addi a2, s0, -1               # argomento = tmp - 1
      jal  FACT                     # chiama funzione FATT
      mul  s0, s0, a0                # tmp = tmp * fatt (tmp-1)
      mv   a0, s0                    # prepara valore di uscita
ENDIF: ld  s0, 0(sp)                 # ripristina ra (callee-saved)
      ld   ra, 8(sp)                 # ripristina s0 (callee-saved)
      addi sp, sp, 16                # toglie area di 16 byte
      ret                           # rientro a chiamante
```

Qui non è necessario salvare sullo stack a2 prima di usarlo nuovamente per il passaggio parametri, perché l'argomento della funzione non viene utilizzato anche dopo la chiamata ricorsiva



traduzione del fattoriale – ii

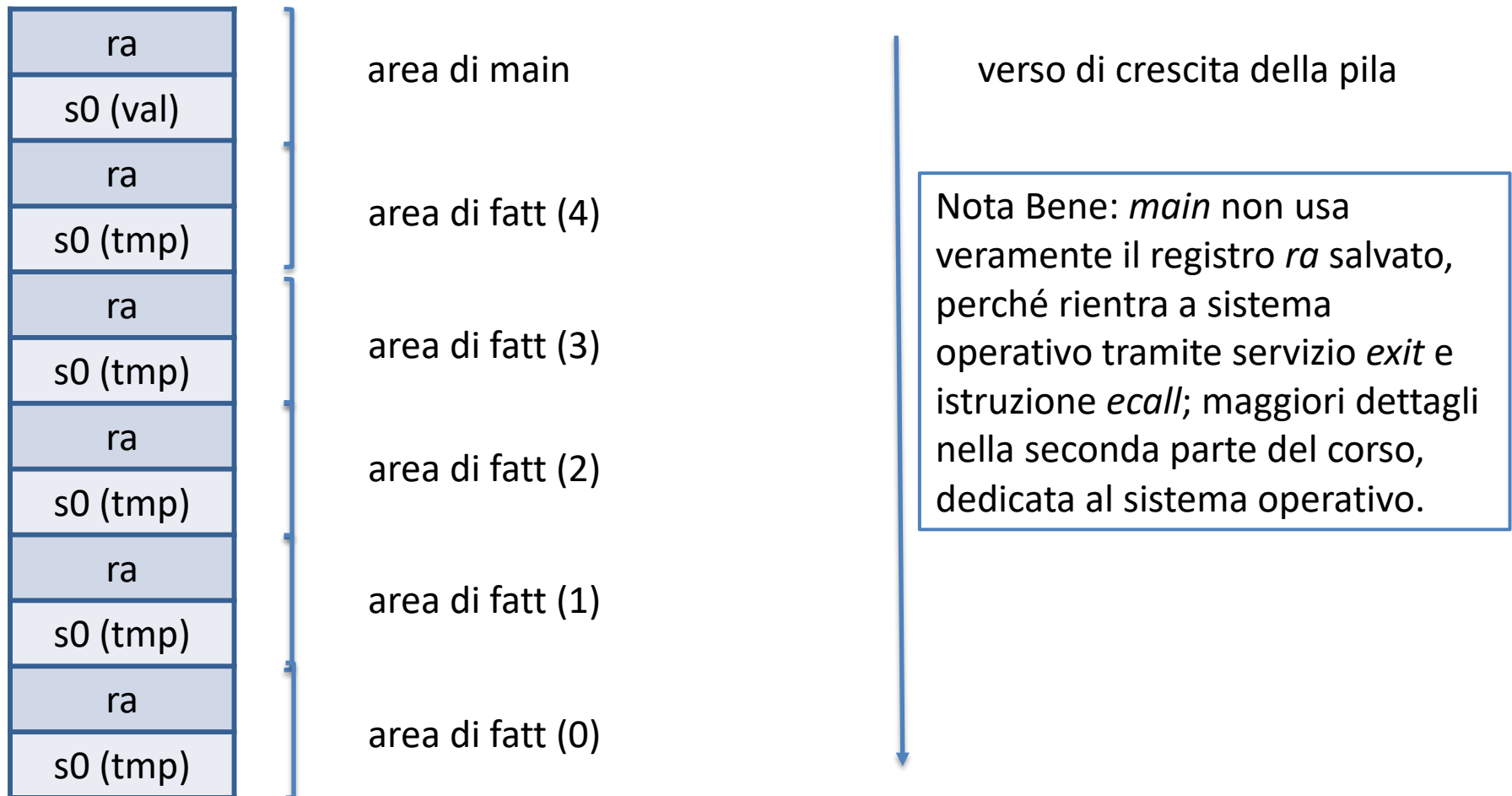
```
.data          # segmento dati (variabili globali)
RET: .dword 0   # variabile globale RET (inizializzata)

.text          # segmento testo (continua)
MAIN: addi sp, sp, -16      # crea area da 16 byte (2 dword)
sd    ra, 8(sp)           # salva reg. ra (callee-saved)
sd    s0, 0(sp)           # salva reg. s0 (callee-saved)
li    s0, 4                # inizializza val = 4
mv    a2, s0
jal   FACT                # chiama funzione FATT
la    t0, RET              # carica indirizzo di varglob RET
sd    a0, 0(t0)           # aggiorna ret = fact (val)
ld    s0, 0(sp)           # ripristina s0 (callee-saved)
ld    ra, 8(sp)           # ripristina ra (callee-saved)
addi  sp, sp, 16          # toglì area da 16 byte (2 dword)
li    a0, 0                # servizio di terminazione (93 = exit)
li    a7, 93               # codice di terminazione (0 = normale)
ecall                          # invoca servizio exit (termina prog.)
```

→ non sarebbe strettamente necessario salvare «ra» per il main, essendo il main una funzione foglia; ma nel farlo si è certamente conservativi – gcc per linux lo farà!



pila – disposizione delle aree di attivazione



valutazione di espressioni algebriche



le espressioni algebriche

i compilatori traducono in linguaggio macchina la valutazione di una espressione algebrica affrontando in **modo ottimizzato** i seguenti passi schematici:

- derivazione dell'albero sintattico dell'espressione
- ordinamento dei nodi dell'albero
- assegnamento dei registri

noi adottiamo una «regola» semplice e intuitiva

che non richiede la derivazione dell'albero sintattico
e non ottimizza la traduzione in modo completo



valutazione di espressioni algebriche

espressione: $a + b - 5 - (d - a) + c$

ma anche: $a + b - (\text{funz}(b - c, d) + c)$

procedimento (sulle slide delle convenzioni ci sono maggiori dettagli)

- completa l'espressione con le parentesi associando da sinistra
- ripeti fino ad avere esaminata tutta l'espressione
 - ✓ valuta il primo operatore valutabile (da sinistra)
 - ✓ un operatore è valutabile se le sue parti sinistra e destra sono:
una variabile oppure una costante oppure una sottoespressione
già calcolata in un registro di tipo t

Nota: le operazioni aritmetiche lavorano solo su registri

- ✓ se una variabile (tipicamente globale) è in memoria prima di usarla deve essere (sempre) caricata in un registro (temporaneo); se è da aggiornare (p.es. per un assegnamento) va (ri)memorizzata
- ✓ se la variabile è associata ad un registro (argomento, variabile locale, ...) si usa il registro che la contiene; se è da aggiornare si modifica il registro



esempio 1 – con variabili globali

espressione:

$$a + b - 5 - (d - a) + c$$

- completa l'espressione con le parentesi associando da sinistra (potremmo anche omettere la parentesi più esterna)

$$(((a + b) - 5) - (d - a)) + c$$

- valuta l'espressione (risultato finale nel registro *t1*)

```
la    t0, A
ld    t1, 0(t0)    # carica A
la    t0, B
ld    t2, 0(t0)    # carica B
add   t1, t1, t2    # ( )
addi  t1, t1, -5    # ( )
la    t0, D
ld    t2, 0(t0)    # carica D
la    t0, A
ld    t3, 0(t0)    # carica A
sub   t2, t2, t3    # ( )
sub   t1, t1, t2    # ( )
la    t0, C
ld    t2, 0(t0)    # carica C
add   t1, t1, t2    # ( )
```

regole implicite utilizzate

- registro *t0* usato ripetutamente per indirizzi di variabili globali
- si riscrive il risultato nel registro con indice minore
- si riutilizzano i registri

ci sono un po' di gradi di libertà nella scelta dei registri e nel fino a che punto riutilizzarli (ottimizzazioni effettuate dal compilatore – qui non interessano)



esempio 2 – con variabili globali e chiamata di funzione

espressione: $a + b - (\text{funz}(b - c, d) + c)$

- completa l'espressione con le parentesi associando da sinistra

$((a + b) - (\text{funz}(b - c, d) + c))$

- valuta l'espressione (ris. in $t1$ – sono sottointese tutte le $la\ t0, A; la\ t0, B; etc...$)

ld	t1, 0(t0)	# carica A
ld	t2, 0(t0)	# carica B
add	t1, t1, t2	# ()
ld	t2, 0(t0)	# carica B
ld	t3, 0(t0)	# carica C
sub	a2, t2, t3	# primo parametro di funz
ld	a3, 0(t0)	# secondo parametro di funz
addi	sp, sp, -8	# push t1
sd	t1, 0(sp)	# salva t1 in pila
jal	FUNZ	# chiama funzione FUNZ
ld	t1, 0(sp)	# ripristina t1 da pila
addi	sp, sp, 8	# pop t1
ld	t2, 0(t0)	# carica C
add	t2, a0, t2	# ()
sub	t1, t1, t2	# ()

Nota Bene: *push* e *pop* di $t1$ per riaverlo inalterato dopo FUNZ.

